

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Develop a Product Billing Calculator Application Using HTML, CSS, and JavaScript

2. AIM

To design and develop a responsive Product Calculator web application that takes product details, quantity, and unit price, and dynamically computes and displays the live total price using JavaScript DOM event listeners.

3. OBJECTIVE

The objectives of this experiment are:

- To build a modern card-based user interface using semantic HTML5 elements.
 - To style input controls and two-column layouts using CSS3 Flexbox.
 - To capture user inputs dynamically using `getElementById()`.
 - To parse numerical input safely using `parseFloat()` with fallback defaults.
 - To prevent negative totals using `Math.max()`.
 - To listen to live input and change events using `addEventListener()`.
 - To format numerical totals into formatted currency strings with `toFixed(2)`.
-

4. THEORY

Live Event Handling — The input event fires synchronously whenever the value of an `<input>` element is modified by typing or using stepper buttons; the change event fires when input focus shifts.

Numerical Parsing — Input values return string types. `parseFloat()` parses strings into floating-point numbers, and `|| 0` ensures safe evaluation if the field is empty or non-numeric.

Boundary Clamping — `Math.max(0, quantity * unitPrice)` guarantees that the calculated total price is non-negative.

Currency Formatting — `toFixed(2)` formats floating point numbers with exactly two decimal digits for currency display.

CSS Flexbox Layout — `display: flex` arranges form fields into horizontal columns with flexible width and consistent spacing.

5. ALGORITHM / PROCEDURE

- Create `index.html` containing a container card with header, input form, and result display section.
 - Add input fields for Product Name, Quantity (`min="0", step="1", default "1"`), and Unit Price (`min="0", step="0.01", default "0.00"`).
 - Add a result container displaying the label "Total Price" and a span with `id="totalDisplay"`.
 - Apply CSS styles for layout centering, input styling, focus effects, and flex row alignment.
 - In JavaScript, select input elements and `totalDisplay` using `getElementById()`.
 - Define `calculateTotal()` to compute quantity multiplied by unit price with zero fallback.
 - Define `updateTotalDisplay()` to format the result as ₹XX.XX and update `totalDisplay.textContent`.
 - Attach input and change event listeners to quantity and price input fields.
 - Call `updateTotalDisplay()` on page load to initialize the total display.
 - Open `index.html` in a web browser, test calculations with various values, and verify results.
-

6. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product Calculator</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: "Segoe UI", system-ui, sans-serif;
      background: #f0f2f5;
      color: #1a1a1a;
      min-height: 100vh;
```

```
display: flex;
align-items: center;
justify-content: center;
padding: 24px;
}
.calculator {
background: #ffffff;
width: 100%;
max-width: 420px;
border-radius: 12px;
box-shadow: 0 4px 24px rgba(0, 0, 0, 0.08);
overflow: hidden;
}
.calculator-header {
background: #1e3a5f;
color: #ffffff;
padding: 20px 24px;
}
.calculator-header h1 { font-size: 1.25rem; font-weight: 600; }
.calculator-header p { font-size: 0.85rem; opacity: 0.85; margin-top: 4px; }
.calculator-body { padding: 24px; }
.form-group { margin-bottom: 18px; }
.form-group label {
display: block;
font-size: 0.8rem;
font-weight: 600;
color: #555;
margin-bottom: 6px;
text-transform: uppercase;
}
.form-group input {
width: 100%;
padding: 11px 14px;
font-size: 0.95rem;
border: 1px solid #d0d5dd;
border-radius: 8px;
background: #fafbfc;
}
.form-group input:focus {
outline: none;
```

```

border-color: #1e3a5f;
box-shadow: 0 0 3px rgba(30, 58, 95, 0.12);
background: #ffffff;
}
.row { display: flex; gap: 14px; }
.row .form-group { flex: 1; }
.result {
margin-top: 8px;
padding: 16px 18px;
background: #f0f7ff;
border: 1px solid #c5dbf5;
border-radius: 8px;
display: flex;
justify-content: space-between;
align-items: center;
}
.result-label { font-size: 0.9rem; font-weight: 500; color: #334155; }
.result-value { font-size: 1.35rem; font-weight: 700; color: #1e3a5f; }
</style>
</head>
<body>

<div class="calculator">
  <div class="calculator-header">
    <h1>Product Calculator</h1>
    <p>Enter details below</p>
  </div>

  <div class="calculator-body">
    <div class="form-group">
      <label for="productName">Product Name</label>
      <input type="text" id="productName" placeholder="e.g. Wireless Mouse">
    </div>

    <div class="row">
      <div class="form-group">
        <label for="quantity">Quantity</label>
        <input type="number" id="quantity" min="0" step="1" value="1">
      </div>
      <div class="form-group">

```

```
<label for="price">Unit Price (₹)</label>
<input type="number" id="price" min="0" step="0.01" value="0.00">
</div>
</div>

<div class="result">
  <span class="result-label">Total Price</span>
  <span class="result-value" id="totalDisplay">₹0.00</span>
</div>
</div>
</div>

<script>
const productNameInput = document.getElementById("productName");
const quantityInput = document.getElementById("quantity");
const priceInput = document.getElementById("price");
const totalDisplay = document.getElementById("totalDisplay");

function calculateTotal() {
  const quantity = parseFloat(quantityInput.value) || 0;
  const unitPrice = parseFloat(priceInput.value) || 0;
  return Math.max(0, quantity * unitPrice);
}

function updateTotalDisplay() {
  const total = calculateTotal();
  totalDisplay.textContent = "₹" + total.toFixed(2);
}

quantityInput.addEventListener("input", updateTotalDisplay);
priceInput.addEventListener("input", updateTotalDisplay);
quantityInput.addEventListener("change", updateTotalDisplay);
priceInput.addEventListener("change", updateTotalDisplay);

updateTotalDisplay();
</script>

</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<code>parseFloat(input.value) 0</code>	Parses input string to float; defaults to 0 if input is empty or invalid.
<code>Math.max(0, quantity * unitPrice)</code>	Computes product total while preventing negative numerical results.
<code>total.toFixed(2)</code>	Formats numerical result with exactly two fixed decimal places.
<code>addEventListener("input", ...)</code>	Listens for immediate keystrokes or value adjustments on input fields.
<code>addEventListener("change", ...)</code>	Listens for committed value changes when focus shifts from inputs.
<code>display: flex; gap: 14px;</code>	Aligns quantity and price input boxes side-by-side using CSS Flexbox.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load page with default inputs	Quantity = 1, Price = 0.00; Total displays ₹0.00	₹0.00 displayed on load	Pass
TC-02	Enter Quantity = 2, Price = 750.50	Total updates in real time to ₹1501.00	₹1501.00 calculated live	Pass
TC-03	Enter Quantity = 3, Price = 49.99	Total calculates accurately to ₹149.97	₹149.97 displayed	Pass
TC-04	Clear quantity input (empty field)	Evaluates as 0; Total displays ₹0.00	Fallback to 0 handled	Pass
TC-05	Increment quantity using number spinner	Total recalculates immediately on each click	Live recalculation verified	Pass

9. OUTPUT

Output 1: Load page with default inputs

The Product Calculator card renders with default Quantity = 1, Unit Price = 0.00, and Total Price showing ₹0.00.

The image shows a 'Product Calculator' card with a dark blue header. Below the header, the text 'Enter details below' is displayed. The form contains three input fields: 'PRODUCT NAME' with the placeholder 'e.g. Wireless Mouse', 'QUANTITY' with the value '1', and 'UNIT PRICE (₹)' with the value '0.00'. At the bottom, a light blue box displays 'Total Price' as '₹0.00'.

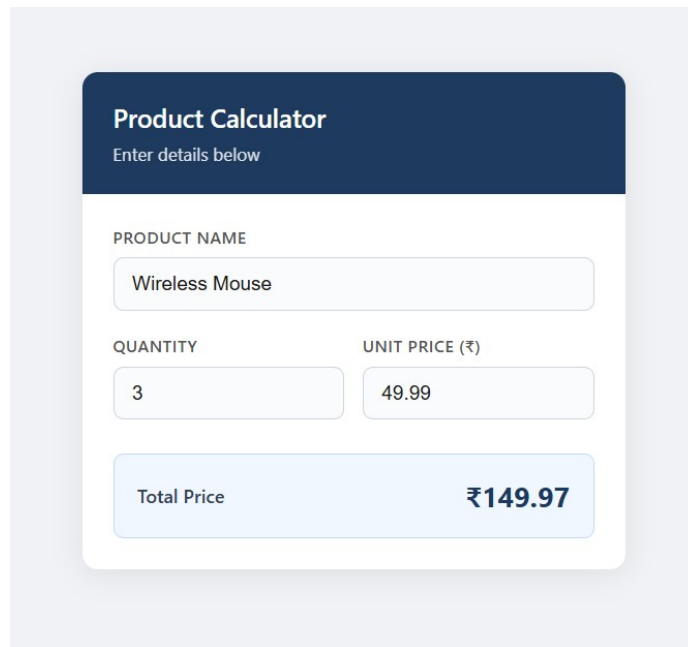
Output 2: Enter Quantity = 2, Price = 750.50

When entering Product Name "Wireless Mouse", Quantity "2", and Unit Price "750.50", the Total Price updates dynamically to ₹1501.00.

The image shows the 'Product Calculator' card with updated inputs. The 'PRODUCT NAME' field now contains 'Wireless Mouse'. The 'QUANTITY' field contains '2', and the 'UNIT PRICE (₹)' field contains '750.50'. The 'Total Price' at the bottom has updated to '₹1501.00'.

Output 3: Enter Quantity = 3, Price = 49.99

Using the input spinner arrows to increase quantity from 2 to 3 and decrease price to 49.99 immediately updates the Total Price to ₹149.97 .

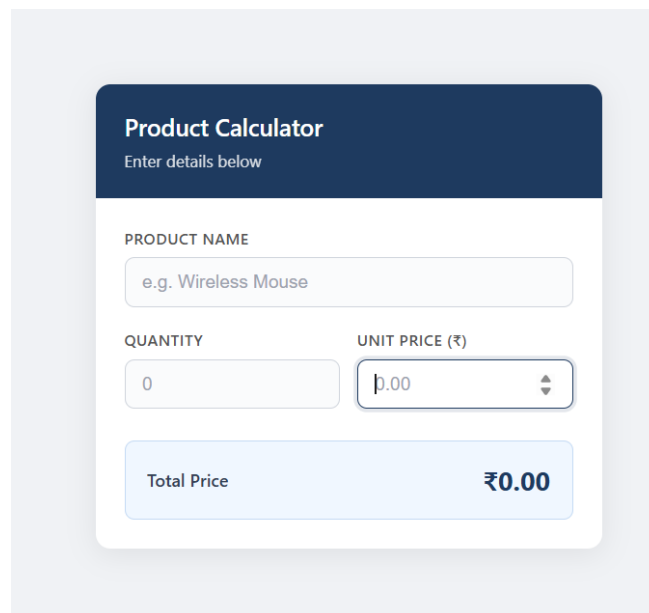


The image shows a 'Product Calculator' web form. At the top, a dark blue header contains the title 'Product Calculator' and the instruction 'Enter details below'. Below this, the form has three input fields: 'PRODUCT NAME' with the value 'Wireless Mouse', 'QUANTITY' with the value '3', and 'UNIT PRICE (₹)' with the value '49.99'. At the bottom, a light blue box displays 'Total Price' as '₹149.97'.

PRODUCT NAME	QUANTITY	UNIT PRICE (₹)	Total Price
Wireless Mouse	3	49.99	₹149.97

Output 4: Clear quantity input (empty field)

Clear the Inputs The Total Price Evaluates as 0 and Total displays ₹0.00.

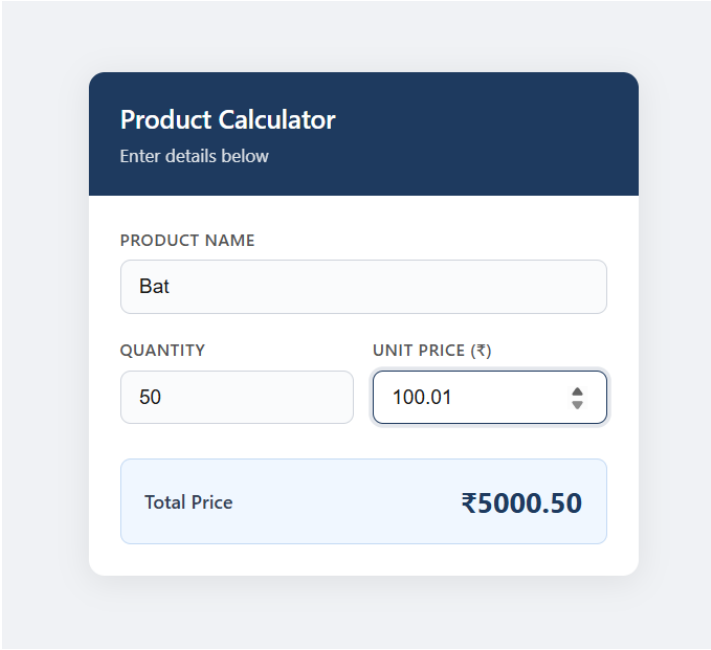


The image shows the 'Product Calculator' web form with the inputs cleared. The 'PRODUCT NAME' field now contains the placeholder text 'e.g. Wireless Mouse'. The 'QUANTITY' field is empty, showing '0', and the 'UNIT PRICE (₹)' field is also empty, showing '0.00'. The 'Total Price' at the bottom is now '₹0.00'.

PRODUCT NAME	QUANTITY	UNIT PRICE (₹)	Total Price
e.g. Wireless Mouse	0	0.00	₹0.00

Output 5: Increment quantity using number spinner

Total recalculates immediately on each click.



The screenshot displays a web application titled "Product Calculator" with the instruction "Enter details below". It features three input fields: "PRODUCT NAME" with the value "Bat", "QUANTITY" with the value "50", and "UNIT PRICE (₹)" with the value "100.01". The "UNIT PRICE (₹)" field is a number spinner. Below these fields, a "Total Price" label is followed by the calculated value "₹5000.50".

PRODUCT NAME	QUANTITY	UNIT PRICE (₹)	Total Price
Bat	50	100.01	₹5000.50

10. RESULT

Thus, the Product Calculator web application was successfully designed and developed using HTML5, CSS3, and JavaScript. The application demonstrates dynamic numerical input parsing, real-time event listening, boundary protection, and currency formatting. All test cases executed with expected results.